

Souvenirs

Amaru está comprando recuerdos en una tienda extranjera. Hay N **tipos** de recuerditos. En la tienda hay disponibles una cantidad suficiente de recuerditos de cada tipo.

Cada tipo de recuerdito tiene un precio fijo. Es decir, un recuerdito de tipo i ($0 \leq i < N$) tiene el precio de $P[i]$ monedas, donde $P[i]$ es un número entero positivo.

Amaru sabe que los tipos de recuerditos están ordenados por precio de forma decreciente, y que el precio de cada recuerdito es distinto. En concreto, $P[0] > P[1] > \dots > P[N-1] > 0$. Además, él fue capaz de aprenderse el valor de $P[0]$. Lamentablemente, Amaru no dispone de más información sobre el precio de cada recuerdito.

Para comprar algunos recuerditos, Amaru realizará una serie de transacciones con el vendedor. Cada transacción consta de los siguientes pasos:

1. Amaru entrega una cantidad positiva de monedas al vendedor.
2. El vendedor pone estas monedas en un montón sobre la mesa en la habitación de atrás, donde Amaru no puede verlas.
3. El vendedor considera cada tipo de recuerdito de $0, 1, \dots, N-1$ en ese orden, uno por uno. Cada tipo de recuerdito se considera **exactamente una vez** por transacción.
 - Al considerar el recuerdito de tipo i , si el número actual de monedas en el montón es al menos $P[i]$, entonces
 - El vendedor quita $P[i]$ monedas del montón, y
 - El vendedor pone un recuerdito de tipo i en la mesa.
4. El vendedor le da a Amaru todas las monedas que quedan en el montón y todos los recuerditos que hay sobre la mesa.

Ten en cuenta que no hay monedas ni recuerditos en la mesa antes de iniciar cada nueva transacción. Tu tarea es pedirle a Amaru que realice una cierta cantidad de transacciones, de modo que:

- En cada transacción él compre **al menos un** recuerdito, y
- en general él compre **exactamente** i recuerditos del tipo i , para cada i tal que $0 \leq i < N$.

Ten en cuenta que esto significa que Amaru no debe comprar ningún recuerdito del tipo 0.

Amaru no tiene por qué minimizar el número de transacciones y tiene un suministro ilimitado de monedas.

Detalles de Implementación

Debes implementar la siguiente función:

```
void buy_souvenirs(int N, long long P0)
```

- N : el número de tipos de recuerdos.
- $P0$: el valor de $P[0]$.
- Esta función se llama exactamente una vez para cada caso de prueba.

La función anterior puede realizar llamadas a la siguiente función para indicarle a Amaru que realice una transacción:

```
std::pair<std::vector<int>, long long> transaction(long long M)
```

- M : el número de monedas entregadas al vendedor por Amaru.
- La función devuelve un pair. El primer elemento del pair es un arreglo L , que contiene los tipos de recuerdos que se han comprado (en orden creciente). El segundo elemento es un entero R , la cantidad de monedas devueltas a Amaru después de la transacción.
- Es necesario que $P[0] > M \geq P[N - 1]$. La condición $P[0] > M$ asegura que Amaru no compra ningún recuerdo del tipo 0, y $M \geq P[N - 1]$ se asegura de que Amaru compre al menos un recuerdo. Si no se cumplen estas condiciones, tu solución recibirá el veredicto `Output isn't correct: Invalid argument`. Ten en cuenta que, contrariamente a $P[0]$, el valor de $P[N - 1]$ no se proporciona en la entrada.
- La función puede llamarse como máximo 5000 veces en cada caso de prueba.

El comportamiento del evaluador no es **adaptativo**. Esto significa que la secuencia de precios P es fija antes de llamar `buy_souvenirs`.

Consideraciones

- $2 \leq N \leq 100$
- $1 \leq P[i] \leq 10^{15}$ para cada i para cada $0 \leq i < N$.
- $P[i] > P[i + 1]$ para cada i tal que $0 \leq i < N - 1$.

Subtareas

Subtarea	Puntaje	Consideraciones adicionales
1	4	$N = 2$
2	3	$P[i] = N - i$ para cada i tal que $0 \leq i < N$.
3	14	$P[i] \leq P[i + 1] + 2$ para cada i tal que $0 \leq i < N - 1$.
4	18	$N = 3$
5	28	$P[i + 1] + P[i + 2] \leq P[i]$ para cada i tal que $0 \leq i < N - 2$. $P[i] \leq 2 \cdot P[i + 1]$ para cada i tal que $0 \leq i < N - 1$.
6	33	Sin consideraciones adicionales.

Ejemplo

Considera la siguiente llamada.

```
buy_souvenirs(3, 4)
```

Existen $N = 3$ tipos de recuerdos y $P[0] = 4$. Observa que sólo hay tres posibles secuencias de precios P : $[4, 3, 2]$, $[4, 3, 1]$, y $[4, 2, 1]$.

Supongamos que `buy_souvenirs` llama a `transaction(2)`. Supongamos que la llamada a la función devuelve $([2], 1)$, lo que significa que Amaru compró un recuerdo de tipo 2 y el vendedor le devolvió 1 moneda. Observa que esto nos permite deducir que $P = [4, 3, 1]$, porque:

- Para $P = [4, 3, 2]$, `transaction(2)` habría devuelto $([2], 0)$.
- Para $P = [4, 2, 1]$, `transaction(2)` habría devuelto $([1], 0)$.

Entonces `buy_souvenirs` puede llamar `transaction(3)`, que devuelve $([1], 0)$, lo que significa que Amaru compró un recuerdo de tipo 1 y el vendedor le devolvió 0 monedas. Hasta ahora, en total, ha comprado un recuerdo del tipo 1 y un recuerdo del tipo 2.

Finalmente, `buy_souvenirs` puede llamar `transaction(1)`, que devuelve $([2], 0)$, lo que significa que Amaru compró un recuerdo de tipo 2. Ten en cuenta que también podríamos haber utilizado `transaction(2)` aquí. Para este momento, en total Amaru tiene un recuerdo del tipo 1 y dos recuerdos del tipo 2, según lo requiera.

Evaluador de Ejemplo

Formato de entrada:

```
N
P[0] P[1] ... P[N-1]
```

Formato de salida:

```
Q[0] Q[1] ... Q[N-1]
```

Donde $Q[i]$ es el numero de recuerdos del tipo i comprados en total para cada i tal que $0 \leq i < N$.